

Pr. Zakaria Boulouard

SPARK



Hautes Etudes Commerciales
Depuis 1988 **MAROC**



Objectifs du chapitre

- Dans ce chapitre, on va :
 - Présenter le concept derrière Spark
 - Apprendre à utiliser Spark
 - Introduire le Resilient Distributed Dataset (RDD)
 - Manipuler PySpark
 - Découvrir PySpark MLlib



Introduction

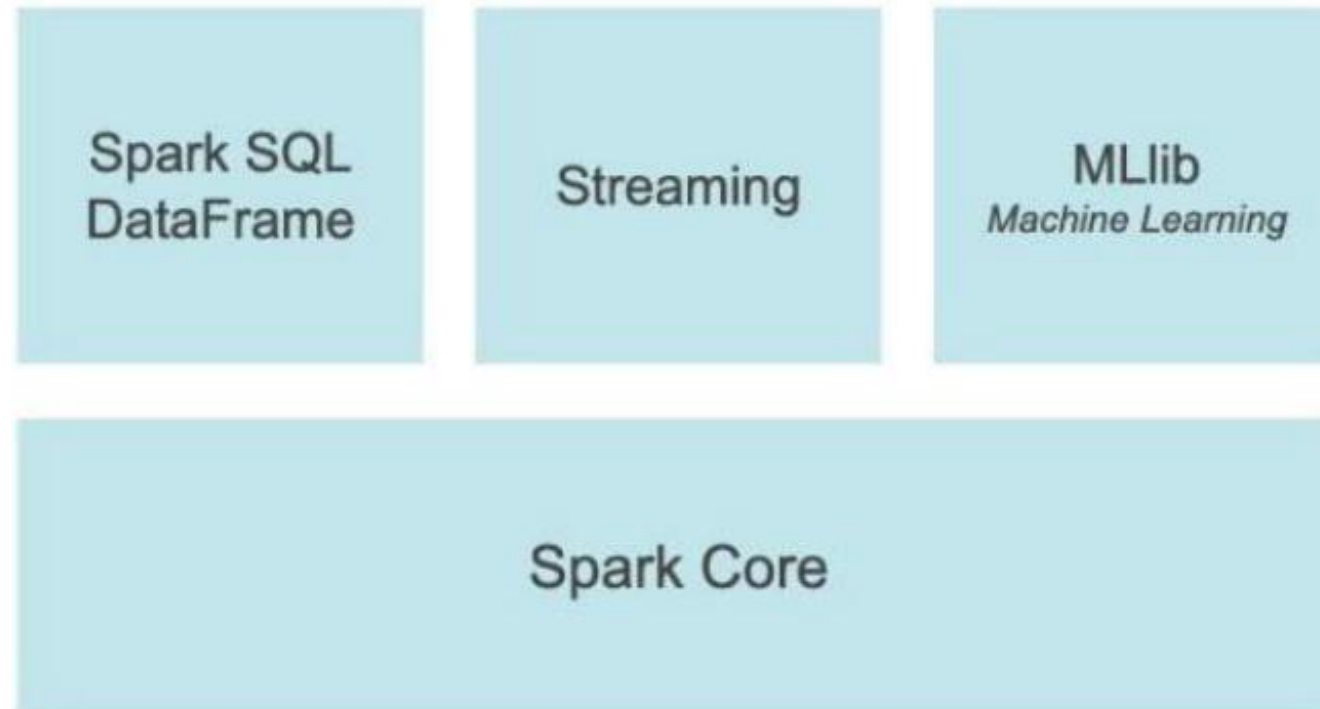
- Apache Spark est un moteur d'analyse unifié pour le traitement de données à grande échelle.
- Il fournit des API de haut niveau en Java, Scala, Python et R, ainsi qu'un moteur optimisé prenant en charge les graphiques d'exécution.



Introduction

- Il prend également en charge un riche ensemble d'outils de niveau supérieur, notamment :
 - « Spark SQL » pour SQL et le traitement des données structurées.
 - « MLlib » pour l'apprentissage automatique.
 - « GraphX » pour le traitement des graphes.
 - « Structured Streaming » pour le calcul incrémentiel et le traitement des flux.

Introduction





Introduction

- Apache Spark a débuté en tant que projet de recherche à l'AMPLab de l'UC Berkeley en 2009 et était open source au début de 2010.
- De nombreuses idées derrière le système ont été présentées dans divers documents de recherche au fil des ans.



Introduction

- Après sa sortie, Spark est devenu une vaste communauté de développeurs et a rejoint Apache Software Foundation en 2013.
- Aujourd'hui, le projet est développé en collaboration par une communauté de centaines de développeurs issus de centaines d'organisations.



Introduction

- Il est conçu pour des performances rapides et utilise la RAM pour la mise en cache et le traitement des données.
- Spark exécute différents types de charges de travail Big Data.



Introduction

- Cela inclut le traitement par lots de type MapReduce, ainsi que le traitement de flux en temps réel, l'apprentissage automatique, le calcul de graphes et les requêtes interactives.
- Avec des API de haut niveau faciles à utiliser, Spark peut s'intégrer à de nombreuses bibliothèques différentes, notamment PyTorch et TensorFlow.



Introduction

- Le moteur Spark a été créé pour améliorer l'efficacité de MapReduce et conserver ses avantages.
- Même si Spark n'a pas son système de fichiers, il peut accéder aux données sur de nombreuses solutions de stockage différentes.
- La structure de données utilisée par Spark est appelée « *Resilient Distributed Dataset* », ou « *RDD* ».



Introduction

- Spark est un moteur de traitement rapide et général compatible avec les données Hadoop.
- Il peut s'exécuter dans des clusters Hadoop via YARN ou le mode standalone de Spark, et il peut traiter des données dans HDFS, HBase, Cassandra, Hive et n'importe quel format d'entrée Hadoop.



Introduction

- Il est conçu pour effectuer à la fois un traitement par lots (batch processing, similaire à MapReduce) et de nouvelles charges de travail telles que le streaming, les requêtes interactives et l'apprentissage automatique.
- Spark est considéré souvent comme un nouveau cadre de calcul dans l'écosystème de Hadoop, mais bien sûr, il peut être déployé seul sans Hadoop.



Introduction

- Il existe cinq composants principaux d'Apache Spark :
 - Apache Spark Core : La base de tout le projet. Spark Core est responsable des fonctions nécessaires telles que la planification, la répartition des tâches, les opérations d'entrée et de sortie, la récupération des pannes, etc... D'autres fonctionnalités sont construites par-dessus.



Introduction

- Il existe cinq composants principaux d'Apache Spark :
 - Spark Streaming : Ce composant permet le traitement des flux de données en direct. Les données peuvent provenir de nombreuses sources différentes, notamment Kafka, Kinesis, Flume, etc...
 - Spark SQL : Spark utilise ce composant pour collecter des informations sur les données structurées et sur la manière dont les données sont traitées.



Introduction

- Il existe cinq composants principaux d'Apache Spark :
 - Bibliothèque de Machine Learning (MLlib) : Cette bibliothèque se compose de nombreux algorithmes d'apprentissage automatique. L'objectif de MLlib est l'évolutivité et de rendre l'apprentissage automatique plus accessible.
 - GraphX : Ensemble d'API utilisées pour faciliter les tâches d'analyse de graphes.



Introduction

- PySpark est une interface pour Apache Spark en Python.
- Elle permet non seulement d'écrire des applications Spark à l'aide des API Python, mais fournit également le shell PySpark pour analyser les données de manière interactive dans un environnement distribué.
- PySpark prend en charge la plupart des fonctionnalités de Spark telles que Spark SQL, DataFrame, Streaming, MLlib et Spark Core.



Terminologie

- Apache Spark : Une plate-forme de traitement de données open source conçue pour le traitement à grande vitesse en mémoire. Pensé comme une alternative à MapReduce. Spark peut être une implémentation à un ou plusieurs nœuds.



Terminologie

- Resilient Distributed Dataset (RDD) : C'est un jeu de données immuable. Cela signifie qu'il ne peut pas être modifié au fil du temps. Les RDD peuvent être stockés sur disque ou en mémoire dans un cluster. Ils sont tolérants aux pannes. Les modifications apportées aux RDD sont effectuées avec les opérations de transformation et d'action.
- Partition : Les données peuvent être réparties en unités logiques plus petites. Ces unités sont appelées « partitions ».



Terminologie

- Cluster Manager : Un service externe comme YARN, Apache Mesos ou Spark Standalone Cluster. Les gestionnaires de cluster assurent la surveillance, la planification et la sécurité.
- Worker Node : Les exécuteurs sont des nœuds de travail générés pendant la durée de vie d'une application Spark. Le travail d'exécution sur les données dans le nœud et le rapport au nœud maître.



Terminologie

- Application : Un programme d'utilisateurs construit sur le cluster Spark. Il se compose d'un programme pilote et des exécuteurs sur le cluster.
- Executor : Existent généralement sur un nœud de travail et implémentent des tâches. Ces tâches sont en mémoire ou sur disque. Les tâches sont créées par le pilote et envoyées aux exécuteurs. Lorsque les exécuteurs ont terminé la tâche, ils renvoient les résultats au pilote.



Terminologie

- Driver Program : Un programme qui s'exécute sur un nœud maître. Il crée le `SparkContext`. Ils déclarent des Transformations et des Actions sur le RDD. C'est le processus où s'exécute la fonction `Main()`. Il convertit les processus en tâches qui sont ensuite exécutées sur les exécuteurs.
- Transformations : Les transformations créent de nouveaux RDD en fonction des opérations effectuées sur les RDD existants. Il existe deux types de transformations : étroites et étendues.



Terminologie

- Narrow Transformations : Tous les éléments requis pour calculer une partition vivant dans une partition parent unique d'un RDD.
- Wide Transformations : Tous les éléments requis pour calculer une partition appartiennent à plusieurs partitions d'un RDD.
- Actions : Les actions sont des opérations qui produisent des valeurs non RDD. Ils déclenchent des transformations pour renvoyer des valeurs. Exemples d'actions : compter, collecter, agréger, min, max ou premier.



Spark vs Hadoop : Performance

- En accédant aux données stockées localement sur HDFS, Hadoop améliore les performances globales.
- Cependant, cela ne correspond pas au traitement en mémoire de Spark.

Spark vs Hadoop : Performance

- Selon les affirmations d'Apache, Spark semble être 100 fois plus rapide lors de l'utilisation de la RAM pour le calcul que Hadoop avec MapReduce.
- La domination est de même avec le tri des données sur des disques. Spark était 3 fois plus rapide et nécessitait 10 fois moins de nœuds pour traiter 100 To de données sur HDFS.



Spark vs Hadoop : Performance

- Cette référence a été suffisante pour établir le record du monde en 2014.
- La principale raison de cette suprématie de Spark est qu'il ne lit pas et n'écrit pas de données intermédiaires sur les disques mais utilise de la RAM.



Spark vs Hadoop : Performance

- Hadoop stocke les données sur de nombreuses sources différentes, puis traite les données par lots à l'aide de MapReduce.
- Tout ce qui précède peut positionner Spark comme le gagnant absolu.



Spark vs Hadoop : Performance

- Cependant, si la taille des données est supérieure à la RAM disponible, Hadoop est le choix le plus logique.
- Un autre point à prendre en compte est le coût de fonctionnement de ces systèmes.



Spark vs Hadoop : Coût

- En comparant Hadoop à Spark en termes de coût, nous devons creuser plus loin que le prix du logiciel.
- Les deux plateformes sont open source et entièrement gratuites.
- Néanmoins, les coûts d'infrastructure, de maintenance et de développement doivent être pris en compte pour obtenir un coût total de possession approximatif.



Spark vs Hadoop : Coût

- Le facteur le plus important dans la catégorie des coûts est le matériel sous-jacent dont vous avez besoin pour exécuter ces outils.
- Étant donné que Hadoop s'appuie sur n'importe quel type de stockage sur disque pour le traitement des données, le coût de son fonctionnement est relativement faible.



Spark vs Hadoop : Coût

- D'autre part, Spark dépend des calculs en mémoire pour le traitement des données en temps réel.
- Ainsi, faire tourner des nœuds avec beaucoup de RAM augmente considérablement le coût de possession.



Spark vs Hadoop : Coût

- Une autre préoccupation est le développement d'applications.
- Hadoop existe depuis plus longtemps que Spark et il est moins difficile de trouver des développeurs de logiciels.



Spark vs Hadoop : Coût

- Les points ci-dessus suggèrent que l'infrastructure Hadoop est plus rentable.
- Bien que cette affirmation soit correcte, nous devons nous rappeler que Spark traite les données beaucoup plus rapidement.
- Par conséquent, il faut un plus petit nombre de machines pour effectuer la même tâche.



Spark vs Hadoop : Traitement des Données

- Les deux Frameworks traitent les données de manière assez différente.
- Bien que Hadoop avec MapReduce et Spark avec RDD traitent les données dans un environnement distribué, Hadoop est plus adapté au traitement par lots.
- En revanche, Spark brille par son traitement en temps réel.



Spark vs Hadoop : Traitement des Données

- Le pipeline de traitement des données sous Hadoop est comme suit :
 1. Lecture des données depuis HDFS.
 2. Génération des couples clé-valeur dans l'étape Map.
 3. Tri des couples avec Shuffle & Sort (étape très coûteuse en termes d'I/O vu que le tri nécessite une écriture sur HDFS).



Spark vs Hadoop : Traitement des Données

- Le pipeline de traitement des données sous Hadoop est comme suit :
 4. Regroupement des résultats dans l'étape Reduce.
 5. Stockage du résultat final sur HDFS.
 6. Retour à l'étape 1 pour un nouveau traitement, etc...



Spark vs Hadoop : Traitement des Données

- Le pipeline de traitement des données sous Spark est comme suit :
 1. Lecture des données depuis HDFS.
 2. Chaque transformation (`map`, `reduceByKey`, `filter`) produit un RDD conservé en RAM, donc moins de latence et presque aucune utilisation de disque.



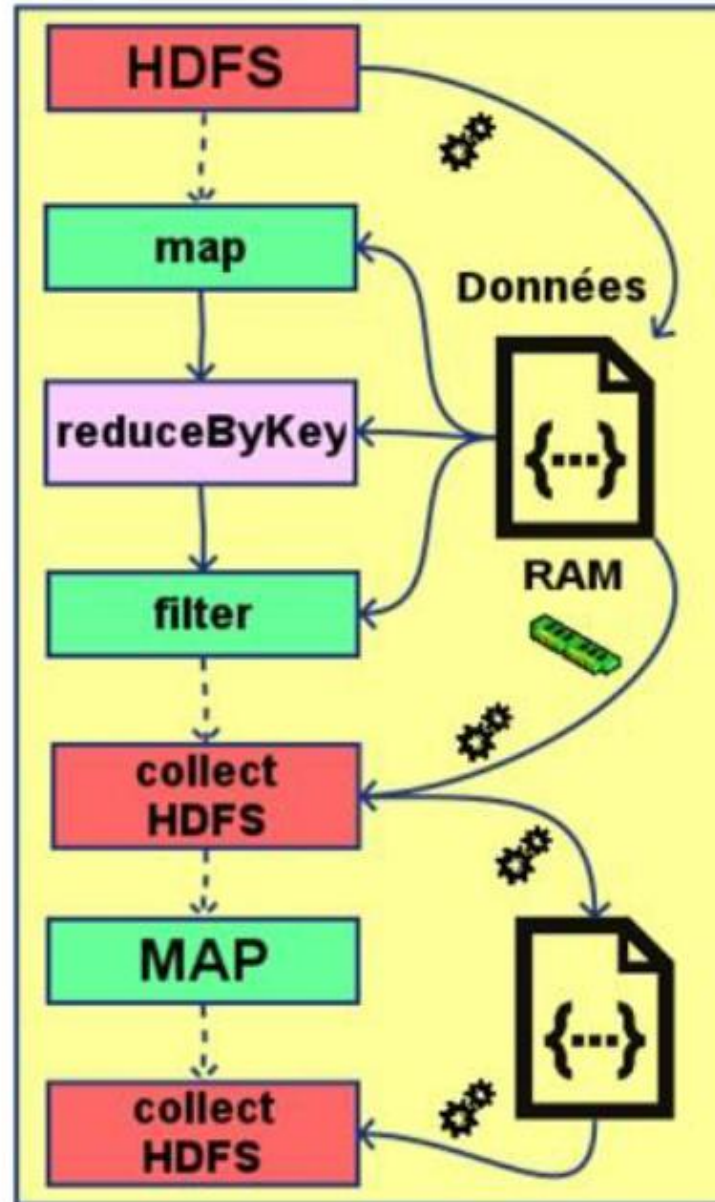
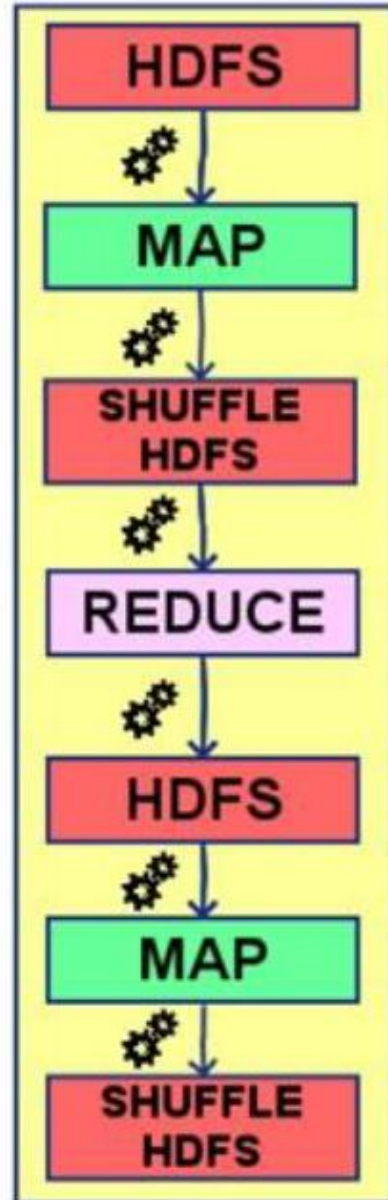
Spark vs Hadoop : Traitement des Données

- Le pipeline de traitement des données sous Spark est comme suit :
 3. Pas d'écriture inutile sur HDFS vu que les données restent en RAM pendant tout le pipeline jusqu'à ce qu'une action (`collect`, `save`, `count`, etc...) force une sortie.
 4. Spark crée un DAG (Directed Acyclic Graph) qui regroupe les transformations pour optimiser l'exécution.



Spark vs Hadoop : Traitement des Données

- Le pipeline de traitement des données sous Spark est comme suit :
 5. Ecriture sur HDFS uniquement sur demande avec une fonction telle que `saveAsTextFile()`, `collect()`, etc...





Spark vs Hadoop : Tolérance aux Pannes

- En parlant de Hadoop contre Spark dans la catégorie de tolérance aux pannes, nous pouvons dire que les deux offrent un niveau suffisant de gestion des échecs.
- De plus, nous pouvons dire que leur approche de la tolérance aux pannes est différente.



Spark vs Hadoop : Tolérance aux Pannes

- Hadoop a la tolérance aux pannes comme base de son fonctionnement.
- Il réplique les données plusieurs fois sur les nœuds.
- En cas de problème, le système reprend le travail en créant les blocs manquants à partir d'autres emplacements.



Spark vs Hadoop : Tolérance aux Pannes

- Les nœuds maîtres suivent l'état de tous les nœuds esclaves.
- Enfin, si un nœud esclave ne répond pas aux pings d'un maître, le maître affecte les travaux en attente à un autre nœud esclave.



Spark vs Hadoop : Tolérance aux Pannes

- Spark utilise des blocs RDD pour atteindre la tolérance aux pannes.
- Le système suit la façon dont l'ensemble de données immuable est créé.
- Ensuite, il peut redémarrer le processus en cas de problème.



Spark vs Hadoop : Tolérance aux Pannes

- Spark peut reconstruire les données dans un cluster en utilisant le suivi DAG des workflows.
- Cette structure de données permet à Spark de gérer les pannes dans un écosystème de traitement de données distribué.



Spark vs Hadoop : Scalabilité

- La frontière entre Hadoop et Spark devient floue dans cette section.
- Hadoop utilise HDFS pour gérer le Big Data.
- Lorsque le volume de données augmente rapidement, Hadoop peut rapidement évoluer pour répondre à la demande.



Spark vs Hadoop : Scalabilité

- Étant donné que Spark n'a pas son système de fichiers, il doit s'appuyer sur HDFS lorsque les données sont trop volumineuses pour être gérées.
- Les clusters peuvent facilement s'étendre et augmenter la puissance de calcul en ajoutant plus de serveurs au réseau.



Spark vs Hadoop : Scalabilité

- En conséquence, le nombre de nœuds dans les deux Frameworks peut atteindre des milliers.
- Il n'y a pas de limite ferme au nombre de serveurs que vous pouvez ajouter à chaque cluster et à la quantité de données que vous pouvez traiter.



Spark vs Hadoop : Scalabilité

- Certains des chiffres confirmés comportent 8 000 machines dans un environnement Spark avec des pétaoctets de données.
- Quand on parle de clusters Hadoop, ils sont bien connus pour héberger des dizaines de milliers de machines et près d'un exaoctet de données.



Spark vs Hadoop : Ease of Use

- Spark est peut-être le cadre le plus récent avec moins d'experts disponibles que Hadoop, mais il est connu pour être plus convivial.
- En revanche, Spark prend en charge plusieurs langages en plus du langage natif (Scala) : Java, Python, R et Spark SQL.
- Cela permet aux développeurs d'utiliser le langage de programmation qu'ils préfèrent.



Spark vs Hadoop : Ease of Use

- Le framework Hadoop est basé sur Java, même si les deux langages principaux pour écrire du code MapReduce sont Java ou Python.
- Hadoop n'a pas de mode interactif pour aider les utilisateurs.
- Cependant, il s'intègre aux outils Pig et Hive pour faciliter l'écriture de programmes MapReduce complexes.



Spark vs Hadoop : Ease of Use

- En plus de la prise en charge des API dans plusieurs langues, Spark gagne avec son mode interactif.
- Vous pouvez utiliser le shell Spark pour analyser les données de manière interactive avec Scala ou Python.
- Le shell fournit un retour instantané aux requêtes, ce qui rend Spark plus facile à utiliser que Hadoop MapReduce.



Spark vs Hadoop : Machine Learning

- L'apprentissage automatique est un processus itératif qui fonctionne mieux en utilisant le calcul en mémoire.
- Pour cette raison, Spark s'est avéré être une solution plus rapide dans ce domaine.



Spark vs Hadoop : Machine Learning

- La raison en est que Hadoop MapReduce divise les travaux en tâches parallèles qui peuvent être trop volumineuses pour les algorithmes d'apprentissage automatique.
- Ce processus crée des problèmes de performances d'I/O dans ces applications Hadoop.



Spark vs Hadoop : Machine Learning

- La bibliothèque Mahout est la principale plate-forme d'apprentissage automatique des clusters Hadoop.
- Mahout s'appuie sur MapReduce pour effectuer le regroupement, la classification et la recommandation.
- Samsara a commencé à remplacer ce projet.



Spark vs Hadoop : Machine Learning

- Spark est livré avec une bibliothèque d'apprentissage automatique par défaut, « MLlib ».
- Cette bibliothèque effectue des calculs ML itératifs en mémoire.
- Il comprend des outils pour effectuer la régression, la classification, la persistance, la construction de pipelines, l'évaluation et bien d'autres.



Spark vs Hadoop : Machine Learning

- Spark avec MLlib s'est avéré neuf fois plus rapide qu'Apache Mahout dans un environnement basé sur disque Hadoop.
- Lorsque vous avez besoin de résultats plus efficaces que ce qu'offre Hadoop, Spark est le meilleur choix pour l'apprentissage automatique.



Spark vs Hadoop : When & Where ?

- Hadoop est plus efficace pour les scénarios impliquant les éléments suivants :
 - Traitement d'ensembles de données volumineux dans des environnements où la taille des données dépasse la mémoire disponible.
 - Traitement par lots avec des tâches qui exploitent les opérations de lecture et d'écriture sur disque.



Spark vs Hadoop : When & Where ?

- Hadoop est plus efficace pour les scénarios impliquant les éléments suivants :
 - Construire une infrastructure d'analyse de données avec un budget limité.
 - Terminer des travaux qui ne sont pas urgents.



Spark vs Hadoop : When & Where ?

- Spark est plus efficace pour les scénarios impliquant les éléments suivants :
 - Traiter des chaînes d'opérations parallèles en utilisant des algorithmes itératifs.
 - Obtenir des résultats rapides avec des calculs en mémoire.



Spark vs Hadoop : When & Where ?

- Spark est plus efficace pour les scénarios impliquant les éléments suivants :
 - Analyse des données de flux en temps réel.
 - Traitement graphique parallèle pour modéliser les données.
 - Toutes les applications de ML.



Spark vs Hadoop : Why not both ?

- Dans la plupart des autres applications, Hadoop et Spark fonctionnent mieux ensemble.
- En tant que successeur, Spark n'est pas là pour remplacer Hadoop mais pour utiliser ses fonctionnalités pour créer un nouvel écosystème amélioré.



Spark vs Hadoop : Why not both ?

- En combinant les deux, Spark peut tirer parti des fonctionnalités qui lui manquent, comme un système de fichiers.
- Hadoop stocke une énorme quantité de données à l'aide de matériel abordable et effectue ensuite des analyses, tandis que Spark apporte un traitement en temps réel pour gérer les données entrantes.



Spark vs Hadoop : Why not both ?

- Sans Hadoop, les applications métier peuvent manquer des données historiques cruciales que Spark ne gère pas.
- Dans cet environnement coopératif, Spark tire également parti des avantages de Hadoop en matière de sécurité et de gestion des ressources.
- Avec YARN, le clustering Spark et la gestion des données sont beaucoup plus faciles.



Spark vs Hadoop : Why not both ?

- Avec YARN, le clustering Spark et la gestion des données sont beaucoup plus faciles.
- Il est possible d'exécuter automatiquement des charges de travail Spark à l'aide de toutes les ressources disponibles.



Spark vs Hadoop : Why not both ?

- Cette collaboration fournit les meilleurs résultats en matière d'analyse rétroactive des données transactionnelles, d'analyse avancée et de traitement des données IoT.
- Les créateurs de Hadoop et Spark avaient l'intention de rendre les deux plates-formes compatibles et de produire des résultats optimaux adaptés à toutes les exigences de l'entreprise.



Spark vs Hadoop : Why not both ?

- Cette collaboration fournit les meilleurs résultats en matière d'analyse rétroactive des données transactionnelles, d'analyse avancée et de traitement des données IoT.
- Les créateurs de Hadoop et Spark avaient l'intention de rendre les deux plates-formes compatibles et de produire des résultats optimaux adaptés à toutes les exigences de l'entreprise.



PySpark

- PySpark est l'API Python du moteur de calcul distribué Apache Spark.
- Dans sa version 1.6.0, PySpark permet principalement de manipuler des RDD (Resilient Distributed Datasets), d'utiliser l'API SQL via `SQLContext`, et d'effectuer des traitements distribués en mémoire à grande échelle.



PySpark

- Au niveau de Cloudera Quickstart, PySpark est déjà installé et parfaitement intégré à Hadoop, ce qui permet d'exécuter des traitements directement sur HDFS dans un environnement pédagogique totalement fonctionnel.



PySpark

- Dans Spark 1.6.0, la porte d'entrée de tout programme est le `SparkContext`.
- Le `SparkContext` est l'objet responsable d'établir la connexion au cluster et de coordonner l'ensemble des opérations distribuées.
- Il configure les `Executors`, gère la communication entre le driver et les `workers`, et met en place le graphe des transformations à exécuter.

PySpark

- Le SparkContext est l'interface principale permettant au programme Python de soumettre des tâches distribuées et d'accéder aux fonctionnalités de Spark.
- Il est défini comme suit :

```
from pyspark import SparkContext  
  
sc = SparkContext("local[*]", "MonApplication")
```



PySpark

- Un RDD (Resilient Distributed Dataset) est la structure de données centrale dans Spark 1.6.0.
- Il s'agit d'une collection distribuée, immuable et partitionnée d'éléments pouvant être traités en parallèle.



PySpark

- Un RDD est :
 - Distribué : Les données sont éclatées en partitions réparties sur différents nœuds du cluster.
 - Résilient : Spark reconstruit automatiquement les partitions perdues grâce au graphe d'exécution (DAG).



PySpark

- Un RDD est :
 - Immuable : Toute transformation produit un nouvel RDD, garantissant la cohérence du calcul.
 - Lazy : Les transformations ne sont pas exécutées immédiatement ; elles ne sont évaluées qu'au moment d'une action.
- Ce modèle permet à Spark d'optimiser l'exécution des pipelines complexes avant leur lancement effectif.



PySpark

- Une transformation est une opération appliquée à un RDD qui produit un nouveau RDD.
- Les transformations sont lazy, c'est-à-dire qu'elles construisent le graphe d'exécution sans déclencher de calcul.



PySpark

- Les transformations adoptées par PySpark sont les suivantes :
 - map(f) : Applique la fonction f à chaque élément du RDD.
 - flatMap(f) : Comme map, mais aplatit les séquences résultantes.
 - filter(f) : Sélectionne uniquement les éléments vérifiant la condition f.



PySpark

- Les transformations adoptées par PySpark sont les suivantes :
 - `reduceByKey(f)` : Agrège les valeurs ayant la même clé à l'aide de la fonction de réduction `f`.
 - `sortBy(f)` : Trie les éléments selon une clé calculée par la fonction `f`.
- Chaque transformation représente un nœud dans le DAG de Spark, ce qui permet de comprendre la logique d'exécution distribuée.



PySpark

- Une `Action` est une opération qui force l'exécution du pipeline construit par les transformations.
- En d'autres termes, c'est une instruction qui demande explicitement un résultat concret.



PySpark

- Les actions adoptées par PySpark sont les suivantes :
 - collect() : ramène toutes les données dans le programme Python (attention à la taille).
 - take(n) : extrait les n premiers éléments.
 - count() : calcule le nombre total d'éléments.



PySpark

- Les actions adoptées par PySpark sont les suivantes :
 - `first()` : retourne le premier élément.
 - `saveAsTextFile(path)` : écrit le résultat dans un répertoire HDFS.
- L'action matérialise le passage d'un graphe abstrait à une exécution réelle sur le cluster.

PySpark

- Exemple : On voudrait utiliser PySpark pour reproduire l'exemple de la partie MapReduce où on avait calculé le nombre d'évaluation pour chaque utilisateur. Le résultat devrait être comme ceci :

344	190
346	193
340	44
342	201

```

# -*- coding: utf-8 -*-

# Import du SparkContext (point d'entree de Spark 1.x)
from pyspark import SparkContext

# -----
# 1. Creation du SparkContext
# -----
# Le SparkContext permet d'etablir la connexion avec le cluster Spark.
# Il gere les RDD, la distribution des taches et l'execution.
sc = SparkContext(appName="UserRatingsCount")

# -----
# 2. Fonction MAP : extraire l'identifiant utilisateur (userId)
# -----
# Cette fonction recoit une ligne brute du fichier u.data.
# Elle decoupe la ligne selon la tabulation, recupere le champ userId
# et renvoie la paire (userId, 1) comme dans le mapper de MapReduce.
def mapper(line):
    parts = line.split("\t")      # Separation de la ligne en 4 champs
    user_id = parts[0]           # Recuperation du champ userId
    return (user_id, 1)          # Retour d'une paire (cle, valeur)

```

```

# -----
# 3. Fonction REDUCE : additionner deux valeurs
# -----
# Cette fonction permet a Spark d'additionner les valeurs associees
# au meme utilisateur. Equivalent du reducer en MapReduce.
def reducer(a, b):
    return a + b                # Addition simple des valeurs

# -----
# 4. Fonction FORMATTER : generer une ligne propre
# -----
# Cette fonction transforme une paire (userId, total) en une ligne texte
# de la forme : "userId \t total". Ce format est ideal pour HUE.
def formatter(pair):
    user = pair[0]              # Recuperation de l'utilisateur
    count = pair[1]             # Recuperation du nombre d'evaluations
    return str(user) + "\t" + str(count)

```

```
# -----  
# 5. Chargement du fichier depuis HDFS  
# -----  
# Lecture du fichier u.data stocke dans HDFS. Chaque ligne devient  
# un element du RDD distribue sur les workers du cluster.  
rdd = sc.textFile("hdfs:///user/cloudera/ml-100k/u.data")  
  
# -----  
# 6. Application de la transformation MAP  
# -----  
# Transformation de chaque ligne du RDD en une paire (userId, 1).  
# Aucun calcul n'est execute ici : Spark construit le plan logique.  
user_movie_pairs = rdd.map(mapper)  
  
# -----  
# 7. Application de la transformation REDUCEBYKEY  
# -----  
# ReduceByKey regroupe automatiquement les paires par userId puis  
# applique la fonction reducer pour sommer tous les "1".  
user_movie_count = user_movie_pairs.reduceByKey(reducer)
```

```
# -----  
# 8. Affichage des 10 premiers resultats dans la console  
# -----  
# take(10) est une action : elle declenche l'execution du DAG complet.  
# Seuls les 10 premiers resultats sont affiches dans le terminal.  
for user, count in user_movie_count.take(10):  
    print(str(user) + "\t" + str(count))  
  
# -----  
# 9. Preparation du format texte pour sauvegarde HDFS  
# -----  
# On re-transforme le RDD de paires en RDD de lignes texte propres.  
formatted = user_movie_count.map(formatter)
```

```
# -----  
# 10. Sauvegarde du resultat formate dans HDFS  
# -----  
# Cette action enregistre tout le RDD dans un repertoire HDFS.  
# HUE affichera ces donnees sous forme propre (user \t count).  
formatted.saveAsTextFile(  
    "hdfs:///user/cloudera/output_spark_user_ratings_formatted"  
)  
  
# -----  
# 11. Arret du SparkContext  
# -----  
# Liberation des ressources et fermeture propre du contexte Spark.  
sc.stop()
```